Claude and ALGO2e In-Class Python Code

January 23, 2026 12:44 PM

The following is the result of uploading the ALGO2e in-class example code into Claude (Sonnet 4.5).
Subsequent inputs to Claude use red font.

This is a market-making trading bot, but it has several significant issues that would prevent it from working effectively. Here are the main problems and suggestions for improvement:

Critical Issues

1. Position Calculation is Wrong
return abs(book[0]['position']) + abs(book[1]['position']) + abs(book[2]['position'])
This adds the absolute values of all positions, so you can't distinguish between being net long vs net short. You're always comparing against positive thresholds, which means the short exposure check position > MAX_SHORT_EXPOSURE will never trigger properly.
Fix: Track positions per ticker or calculate net position correctly:

```
# For individual ticker:
def get_position(ticker):
    resp = s.get('http://localhost:9999/v1/securities')
    if resp.ok:
        book = resp.json()
        for security in book:
            if security['ticker'] == ticker:
                return security['position']
    return 0
```

Claude correctly identified the issue we talked about in class.

The fix is partially correct/useful (track net exposure) and partially not as useful (per ticker position limits)

2. No Spread Check You're posting orders at the best bid/ask, which means:

- You're joining the queue, not providing liquidity
- You make no profit (buying at bid, selling at ask means zero spread)
- You should be quoting *inside* or *improving* the spread

Fix: Add a spread and quote competitively:

```
spread = 0.01  # or calculate based on market conditions
buy_price = best_bid_price + spread
sell_price = best_ask_price - spread
```

Claude is incorrect. At least traditionally, adding orders to the book is adding liquidity (more volume for others to trade with). If bid < ask you generate a profit buying at the bid price and selling at the ask price (+ / - rebates / commissions). Quoting inside the NBBO is not always possible (see follow-up input) or desirable (it reduces your bid-ask spread you earn).

The fix is not useful, could lead to you buying at the ask and selling at the bid (reverse market making, guaranteed loss)

3. Order Management is Primitive

- You cancel ALL orders every 0.5 seconds regardless of fill status
- No tracking of order IDs or fill confirmations
- The aggressive cancel-and-replace causes unnecessary market data overhead

4. No Risk Management

- No per-ticker position limits
- No P&L tracking
- No adverse selection protection
- Total position limit is too simple for multi-ticker market making

5. Unused Functions get_time_sales() and get_order_status() are defined but never used.

Claude's observations are correct, but how would you use this information? What changes would you make to the code?

Suggested Improvements

```
def get_position(ticker):
    resp = s.get('http://localhost:9999/v1/securities')
    if resp.ok:
        book = resp.json()
        for security in book:
            if security['ticker'] == ticker:
                return security['position']
    return 0
def main():
    tick, status = get_tick()
    ticker_list = ['CNR', 'RY', 'AC']
```

```
MAX_PER_TICKER = 10000
SPREAD = 0.02  # Adjust based on market

while status == 'ACTIVE':
    for ticker_symbol in ticker_list:
        position = get_position(ticker_symbol)
        best_bid, best_ask = get_bid_ask(ticker_symbol)

        # Cancel existing orders first
        s.post('http://localhost:9999/v1/commands/cancel',
               params={'ticker': ticker_symbol})

        # Only quote if within position limits
        if abs(position) < MAX_PER_TICKER:
            # Buy order (bid)
            if position < MAX_PER_TICKER:
                buy_price = round(best_bid + SPREAD, 2)
                s.post('http://localhost:9999/v1/orders',
                       params={'ticker': ticker_symbol,
                               'type': 'LIMIT',
                               'quantity': ORDER_LIMIT,
                               'price': buy_price,
                               'action': 'BUY'})

            # Sell order (ask)
            if position > -MAX_PER_TICKER:
                sell_price = round(best_ask - SPREAD, 2)
                s.post('http://localhost:9999/v1/orders',
                       params={'ticker': ticker_symbol,
                               'type': 'LIMIT',
                               'quantity': ORDER_LIMIT,
                               'price': sell_price,
                               'action': 'SELL'})

        sleep(0.2)

    tick, status = get_tick()
```

This is a basic framework - a production market maker would need much more sophisticated logic around pricing, inventory management, and risk controls.

There is a total position limit of 25,000 shares (gross limit) in addition to the 25,000 share net limit (maximum long or short position) that is calculated using the total position of all 3 stocks.

Adding information to Claude - the AI tool can only analyze the information it has available and it may not know everything about the case (or task) from an internet search / existing database. You will likely need to provide more information for the RIT cases.

Ah, I see! There are **two separate limits**:

1. **Net limit**: Maximum of +25,000 long OR -25,000 short (across all 3 stocks combined)
2. **Gross limit**: Maximum of 25,000 total shares (sum of absolute positions across all 3 stocks)

So your get_position() function calculating the sum of absolute values is actually correct for the **gross limit** check. However, you still need to track the **net position** separately.

Corrected Analysis

Your current code has these issues:

1. Missing Net Position Check You check position < MAX_LONG_EXPOSURE where position is the gross (always positive), which doesn't make sense. You need to calculate net position:

```
def get_net_position():
    resp = s.get('http://localhost:9999/v1/securities')
    if resp.ok:
```

With better information, Claude has provided a correct analysis and useful solution for the position limits.

(always positive), which doesn't make sense. You need to calculate net position:

```
def get_net_position():
    resp = s.get('http://localhost:9999/v1/securities')
    if resp.ok:
        book = resp.json()
        return book[0]['position'] + book[1]['position'] + book[2]['position']
    return 0
def get_gross_position():
    resp = s.get('http://localhost:9999/v1/securities')
    if resp.ok:
        book = resp.json()
        return abs(book[0]['position']) + abs(book[1]['position']) + abs(book[2]['position'])
    return 0
```

2. Incorrect Logic Your conditions should be:

- For buying: Check that net position < 25,000 AND gross position < 25,000
- For selling: Check that net position > -25,000 AND gross position < 25,000

3. Still No Profitability You're still posting at best_bid (to buy) and best_ask (to sell), which means zero edge.

Fixed Version

```
def get_net_position():
    resp = s.get('http://localhost:9999/v1/securities')
    if resp.ok:
        book = resp.json()
        return sum(item['position'] for item in book)
    return 0
def get_gross_position():
    resp = s.get('http://localhost:9999/v1/securities')
    if resp.ok:
        book = resp.json()
        return sum(abs(item['position']) for item in book)
    return 0
def main():
    tick, status = get_tick()
    ticker_list = ['CNR', 'RY', 'AC']

    while status == 'ACTIVE':
        net_position = get_net_position()
        gross_position = get_gross_position()

        for ticker_symbol in ticker_list:
            best_bid_price, best_ask_price = get_bid_ask(ticker_symbol)

            # Check BOTH limits before buying
            if net_position < MAX_LONG_EXPOSURE and gross_position < 25000:
                resp = s.post('http://localhost:9999/v1/orders',
                              params={'ticker': ticker_symbol,
                                      'type': 'LIMIT',
                                      'quantity': ORDER_LIMIT,
                                      'price': best_bid_price,  # Still problematic!
                                      'action': 'BUY'})

            # Check BOTH limits before selling
            if net_position > MAX_SHORT_EXPOSURE and gross_position < 25000:
                resp = s.post('http://localhost:9999/v1/orders',
                              params={'ticker': ticker_symbol,
                                      'type': 'LIMIT',
```

With better information, Claude has provided a correct analysis and useful solution for the position limits.

The comment on profitability is still incorrect.

```
                    'quantity': ORDER_LIMIT,
                    'price': best_ask_price,  # Still problematic!
                    'action': 'SELL'})

        sleep(0.5)
        s.post('http://localhost:9999/v1/commands/cancel',
            params={'ticker': ticker_symbol})

    tick, status = get_tick()
```

However, you still have the fundamental issue that buying at the best bid and selling at the best ask generates **no profit**. You need to improve the spread to make money as a market maker.

What if the bid-ask spread is $0.01? What if I buy or sell shares because I improved the spread and the price rose while I was short or fell while I was long?

Adding additional information to Claude, in this case challenging the analysis of profitability already provided.

Excellent questions! You've identified the key risks in market making:

1. Tight Spread Problem ($0.01)

If the bid-ask spread is only $0.01, you're right that there's very little room to profit. However:

- **You can't improve the spread** - if bid is $10.00 and ask is $10.01, you can't quote $10.005
- **Joining at best bid/ask** means you're hoping to capture the spread when you get filled on both sides
- With a $0.01 spread and 500 shares per order, your theoretical profit is only **$5 per round trip** (buy 500 @ $10.00, sell 500 @ $10.01)
- But this assumes you get filled on both sides at favorable prices

2. Adverse Selection Risk (Your Main Concern)

This is the **biggest risk in market making**. You're absolutely right:

Scenario 1: Price rises while you're short

- You sell 500 shares @ $10.01
- Price jumps to $10.10 due to news
- Now you need to buy back at $10.10
- **Loss: $45 on 500 shares** (instead of $5 profit)

Scenario 2: Price falls while you're long

- You buy 500 shares @ $10.00
- Price drops to $9.90 due to news
- Now you need to sell at $9.90
- **Loss: $50 on 500 shares** (instead of $5 profit)

These are all good comments by Claude, the challenge is turning these comments into code / strategy

Why Does This Happen?

You get **adversely selected** - informed traders pick off your quotes when they know something you don't:

- News about earnings, mergers, regulatory changes
- Large institutional orders moving the market
- Correlation with other assets (sector rotation, market crash, etc.)

How Real Market Makers Handle This

1. Quote Skewing Based on Inventory

```
# If you're long (have inventory), shade your quotes to sell
if position > 0:
    buy_price = best_bid - 0.01  # Less aggressive on buys
    sell_price = best_ask        # More aggressive on sells
elif position < 0:
    buy_price = best_bid         # More aggressive on buys
    sell_price = best_ask + 0.01  # Less aggressive on sells
else:
    buy_price = best_bid
```

Claude is partially correct, in that you may alter your prices based on inventory (you do not need to be symmetrical). Claude is focussed on prices, you could consider a similar policy with respect to volume (e.g. buy less when you are long / sell more and vice versa when you are short).

Claude is also stuck on the idea of buying / selling at the NBBO, and is not suggesting you initiate your positions with prices

```
    buy_price = best_bid        # More aggressive on buys
    sell_price = best_ask + 0.01  # Less aggressive on sells
else:
    buy_price = best_bid
    sell_price = best_ask
```

is focussed on prices, you could consider a similar policy with respect to volume (e.g. buy less when you are long / sell more and vice versa when you are short).

Claude is also stuck on the idea of buying / selling at the NBBO, and is not suggesting you initiate your positions with prices outside of the NBBO (i.e. bid below the best bid and offer above the best ask). In-class we will likely see orders "pass through" multiple layers / prices in the book, meaning you can buy and sell at prices outside of the NBBO.

2. Widen Spreads During Volatility

```
# If market is moving fast, widen your quotes
time_sales = get_time_sales(ticker)
if high_volatility_detected(time_sales):
    # Pull quotes or widen spread
    return  # Skip this iteration
```

3. Position Limits Per Ticker Don't let one stock dominate your risk:

```
MAX_PER_TICKER = 5000  # Instead of 25000 total
```

A fixed position limit per ticker is unlikely to be optimal, and could change from iteration to iteration.

4. Fast Cancellation When Price Moves Your current approach of canceling every 0.5 seconds actually helps here - you're not leaving stale quotes out.

Yes, but how much time is your algo spending not trading… (e.g. without orders in the book for one or more ticker symbol?)

5. Monitor Time & Sales for Large Trades

```
recent_trades = get_time_sales(ticker)
if sum(recent_trades[-5:]) > 10000:  # Large recent volume
    # Market might be moving, pull quotes
    return
```

A better indicator of the market moving in one direction is the relative frequency of your buy orders being filled vs. your sell orders. If the market price is rising, your sell orders are more likely to be filled than your buy orders, and vice versa for market prices dropping. This is one way to identify the adverse selection problem.

The Reality

With a $0.01 spread and these risks, **market making is extremely difficult** and you need:

- **Very high volume** to make tiny profits add up
- **Excellent risk management** to avoid the occasional big loss wiping out days of profits
- **Speed** to update quotes faster than adverse price movements

Your current strategy of aggressively canceling and re-quoting is actually reasonable for managing adverse selection risk, but the profitability is questionable unless you're doing massive volume.

Bottom line: Yes, you could lose money if the price moves against your position, and those losses can easily exceed your spread profits. That's why professional market makers invest heavily in speed, technology, and sophisticated risk models.